

TDVI: Inteligencia Artificial

Redes Neuronales Convolucionales

Licenciatura en Tecnología Digital

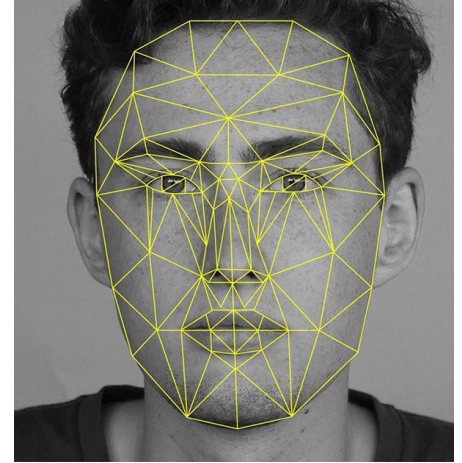
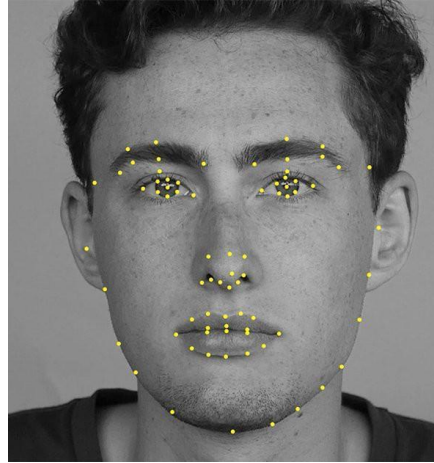


Agenda

1. **Aplicaciones**
2. Convoluciones
3. Ejemplo discreto
4. Convoluciones multidimensionales
5. Completamente conectada (FC) vs. capa convolucional
6. Kernels y Filtros
7. Padding y Stride
8. Pooling
9. Arquitectura de una CNN
10. Bibliografía

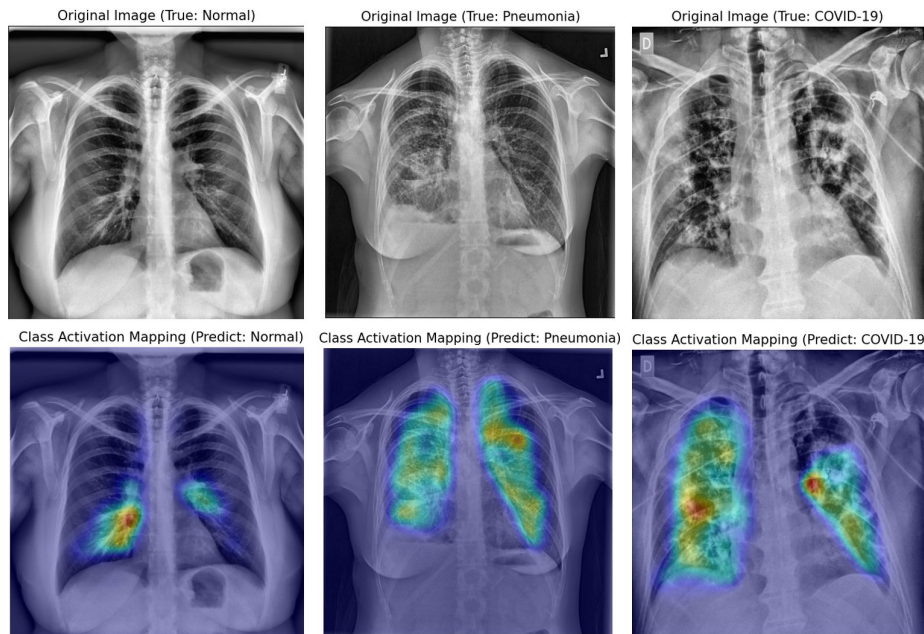
Aplicaciones de redes neuronales

- Autenticación biométrica



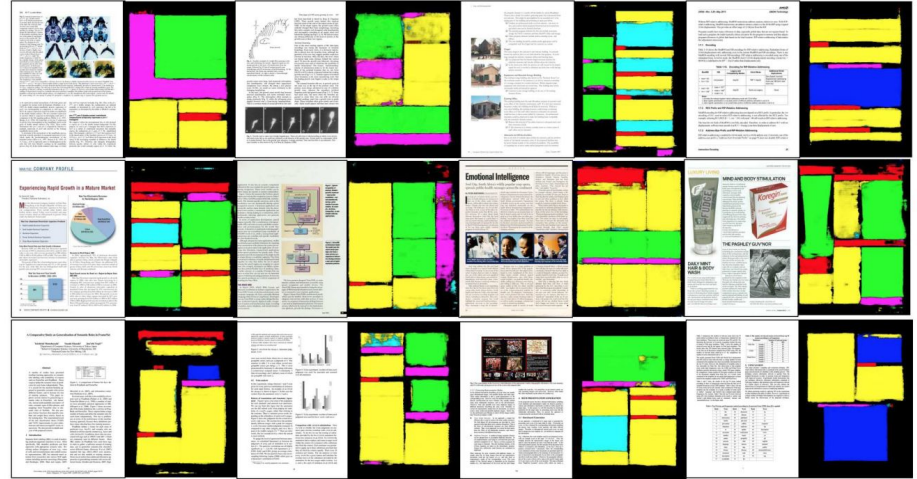
Aplicaciones de redes neuronales

- Autenticación biométrica
- Imágenes médicas



Aplicaciones de redes neuronales

- Autenticación biométrica
- Imágenes médicas
- Análisis de documentos



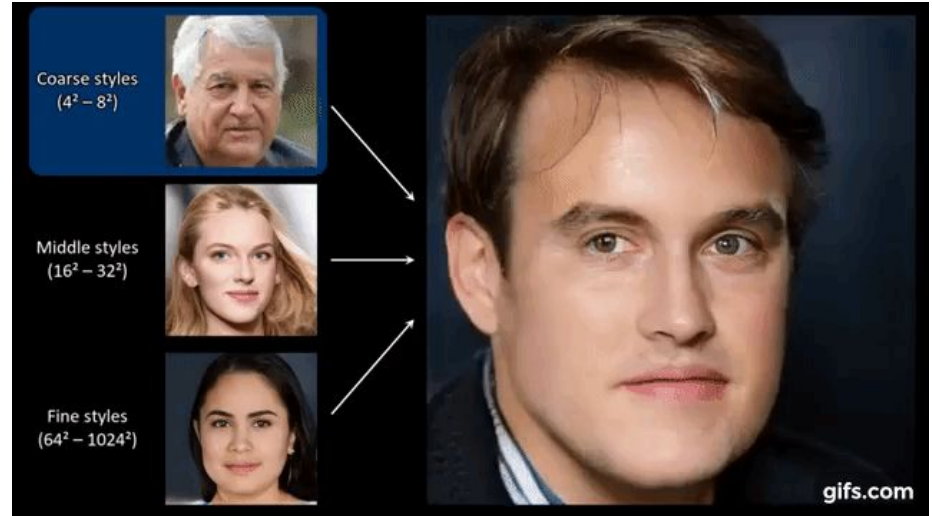
Aplicaciones de redes neuronales

- Autenticación biométrica
- Imágenes médicas
- Análisis de documentos
- Conducción autónoma



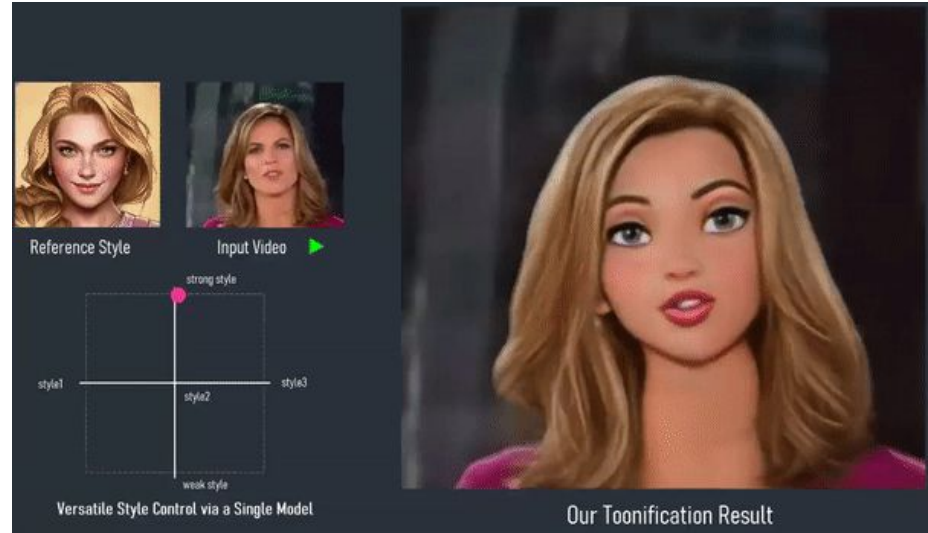
Aplicaciones de redes neuronales

- Autenticación biométrica
- Imágenes médicas
- Análisis de documentos
- Conducción autónoma
- Generación multimedial



Aplicaciones de redes neuronales

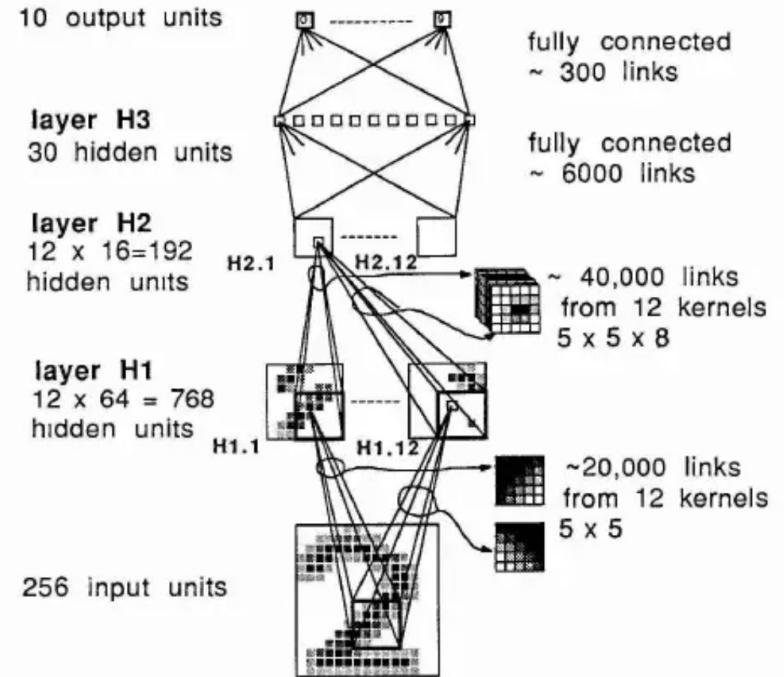
- Autenticación biométrica
- Imágenes médicas
- Análisis de documentos
- Conducción autónoma
- Generación multimedial
- y más...



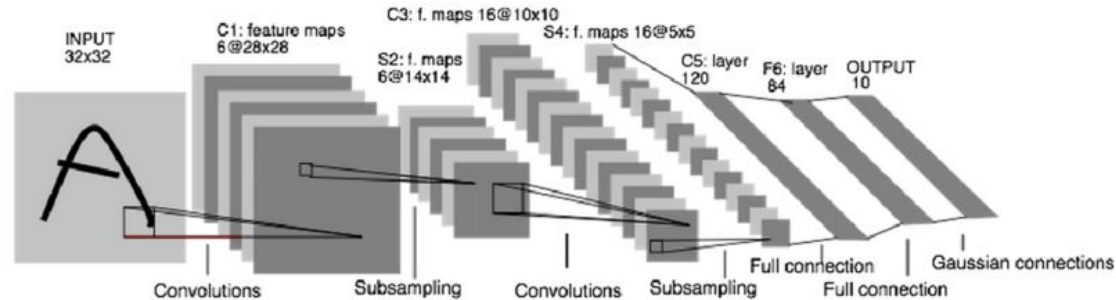
Agenda

1. Aplicaciones
2. **Convoluciones**
3. Ejemplo discreto
4. Convoluciones multidimensionales
5. Completamente conectada (FC) vs. capa convolucional
6. Kernels y Filtros
7. Padding y Stride
8. Pooling
9. Arquitectura de una CNN
10. Bibliografía

Convolutional Neural Nets (CNN): LeNet



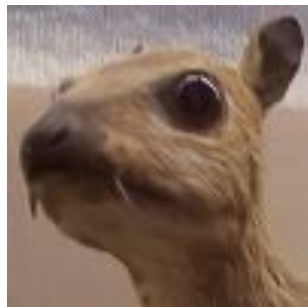
Convolutional Neural Nets (CNN): LeNet



LeNet: a layered model composed of convolution and subsampling operations followed by a holistic representation and ultimately a classifier for handwritten digits. [LeNet]

¿Qué es una convolución?

- En el contexto de procesamiento de imágenes:



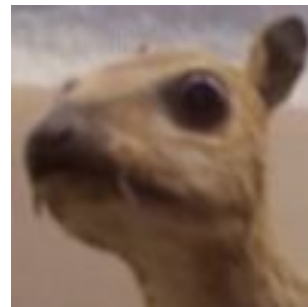
Input



Ridges



Sharpen



Blur

¿Qué es una convolución?

- Una operación matemática que se realiza sobre dos funciones y produce una tercera función
 - expresa cómo la forma de una de ellas se modifica debido a la otra
- Típicamente denotado con un asterisco (*)
- Podemos definir convoluciones en dominios continuos y discretos

$$(f * g)(t) = \int f(a) \cdot g(t - a) da$$

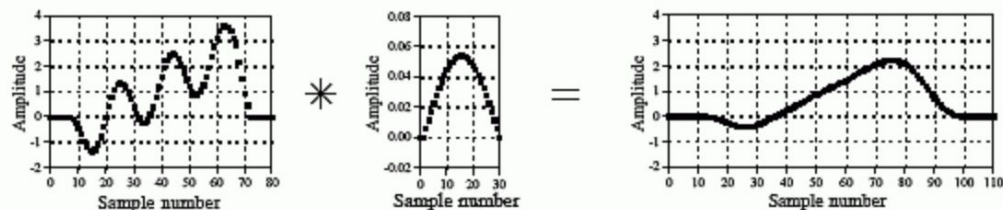
Continuo

$$(f * g)(t) = \sum f(a) \cdot g(t - a)$$

Discreto

¿Qué es una convolución?

a. Low-pass Filter



b. High-pass Filter

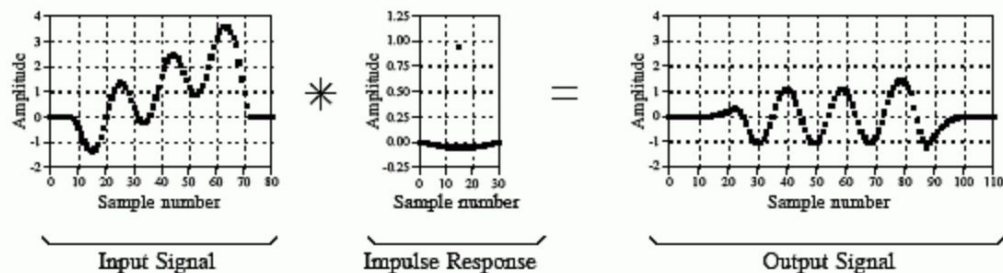
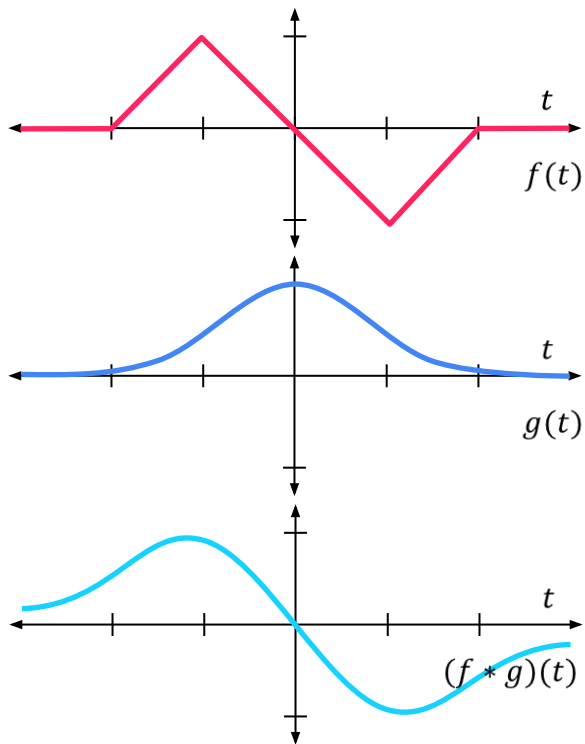


FIGURE 6-3

Examples of low-pass and high-pass filtering using convolution. In this example, the input signal is a few cycles of a sine wave plus a slowly rising ramp. These two components are separated by using properly selected impulse responses.

¿Qué es una convolución?



Continuo

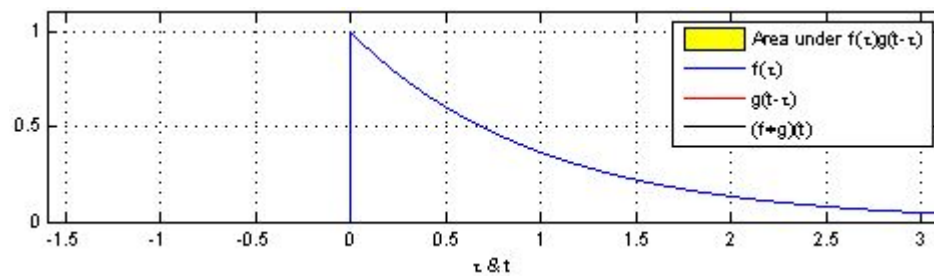
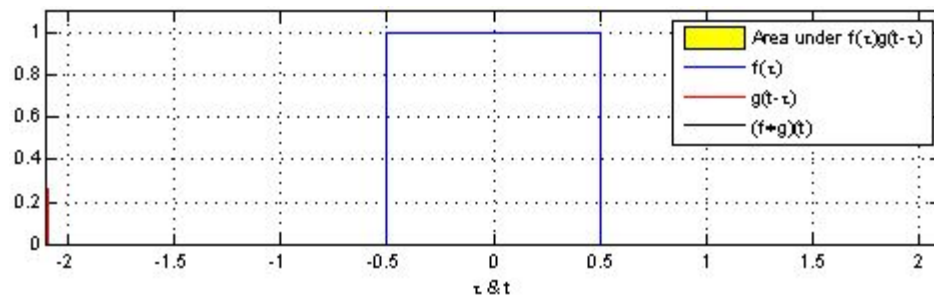
$$p = [1, 2, 3, 4]$$

$$q = [5, 6, 7]$$

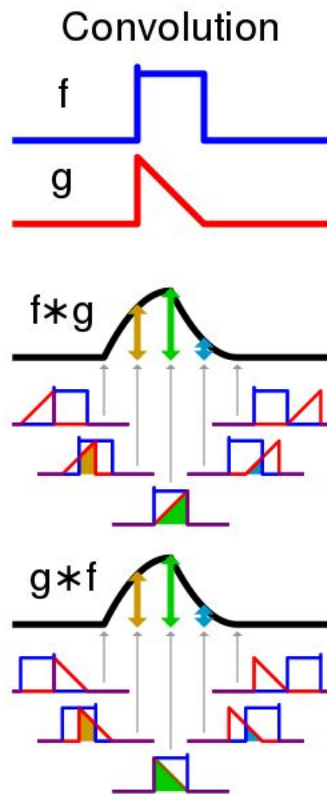
$$p * q = [5, 16, 34, 52, 45, 28]$$

Discreto

¿Qué es una convolución?



¿Qué es una convolución?



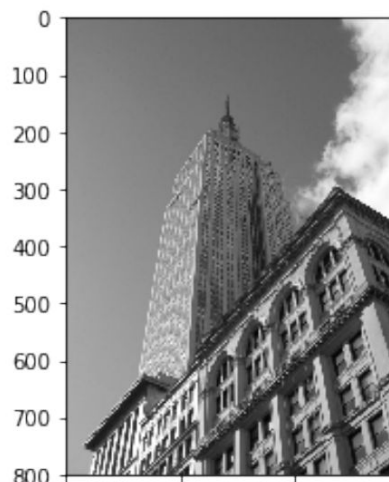
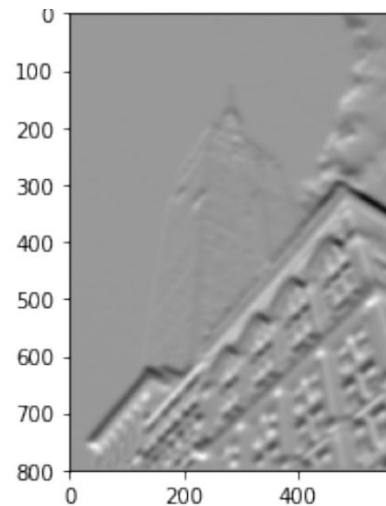


*

-1	0	1
-2	0	2
-1	0	1

=

horizontal edge detector

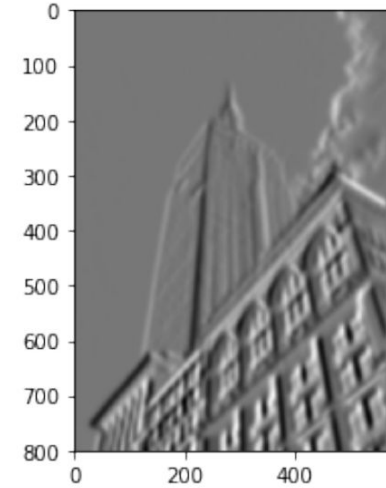


*

-1	-2	-1
0	0	0
1	2	1

=

vertical edge detector



Agenda

1. Aplicaciones
2. Convoluciones
3. **Ejemplo discreto**
4. Convoluciones multidimensionales
5. Completamente conectada (FC) vs. capa convolucional
6. Kernels y Filtros
7. Padding y Stride
8. Pooling
9. Arquitectura de una CNN
10. Bibliografía

Convolución de dos vectores

$$(f * g)(t) = \sum f(a) \cdot g(t - a)$$

$$p = [1, 2, 3, 4]$$

$$q = [5, 6, 7]$$

Convolución de dos vectores

$$(p * q)_n = \sum_{m=-\infty}^{\infty} p_m \cdot q_{n-m}$$

$$p = [1, 2, 3, 4]$$

$$q = [5, 6, 7]$$

Convolución de dos vectores

$$(p * q)_n = \sum_{m=0}^2 p_m \cdot q_{n-m}$$

$$p = [1, 2, 3, 4]$$

$$q = [7, 6, 5]$$



Convolución de dos vectores

$$(p * q)_n = \sum_{m=0}^2 p_m \cdot q_{n-m}$$

$$p = [1, 2, 3, 4]$$

$$q = [7, 6, 5]$$

$$p * q = [...]$$

Convolución de dos vectores

$$(p * q)_n = \sum_{m=0}^2 p_m \cdot q_{n-m}$$

$$p = [1, 2, 3, 4]$$

$$q = [7, 6, 5]$$

$$p * q = [\underline{1 \cdot 5}, \dots]$$

Convolución de dos vectores

$$(p * q)_n = \sum_{m=0}^2 p_m \cdot q_{n-m}$$

$$p = [1, 2, 3, 4]$$

$$q = [7, 6, 5]$$

$$p * q = [5, \underline{1 \cdot 6 + 2 \cdot 5}, \dots]$$

Convolución de dos vectores

$$(p * q)_n = \sum_{m=0}^2 p_m \cdot q_{n-m}$$

$$p = [1, 2, 3, 4]$$

$$q = [7, 6, 5]$$

$$p * q = [5, 16, \underline{1 \cdot 7 + 2 \cdot 6 + 3 \cdot 5}, \dots]$$

Convolución de dos vectores

$$(p * q)_n = \sum_{m=0}^2 p_m \cdot q_{n-m}$$

$$p = [1, 2, 3, 4]$$

$$q = [7, 6, 5]$$

$$p * q = [5, 16, 34, \underline{2 \cdot 7 + 3 \cdot 6 + 4 \cdot 5}, \dots]$$

Convolución de dos vectores

$$(p * q)_n = \sum_{m=0}^2 p_m \cdot q_{n-m}$$

$$p = [1, 2, 3, 4]$$

$$q = [7, 6, 5]$$

$$p * q = [5, 16, 34, 52, \underline{3 \cdot 7 + 4 \cdot 6}, \dots]$$

Convolución de dos vectores

$$(p * q)_n = \sum_{m=0}^2 p_m \cdot q_{n-m}$$

$$p = [1, 2, 3, 4]$$

$$q = [7, 6, 5]$$

$$p * q = [5, 16, 34, 52, 45, \underline{7 \cdot 4}]$$

Convolución de dos vectores

$$(p * q)_n = \sum_{m=0}^2 p_m \cdot q_{n-m}$$

$$p = [1, 2, 3, 4]$$

$$q = [5, 6, 7]$$

$$p * q = [5, 16, 34, 52, 45, 28]$$

Terminología

$$(p * q)_n = \sum_{m=0}^2 p_m \cdot q_{n-m}$$

$p = [1, 2, 3, 4]$ -----► Input

$q = [5, 6, 7]$ -----► Kernel

$p * q = [5, 16, 34, 52, 45, 28]$ -----► Feature map

Terminología

$$(p * q)_n = \sum_{m=0}^2 p_m \cdot q_{n-m}$$

$p = [1, 2, 3, 4]$ -----> Input

$q^1 = [5, 6, 7]$ -----> Filtro 1 - Kernel 1

$q^2 = [7, 6, 5]$ -----> Filtro 2 - Kernel 2

$p * q^1 = [5, 16, 34, 52, 45, 28]$ -----> Feature map 1

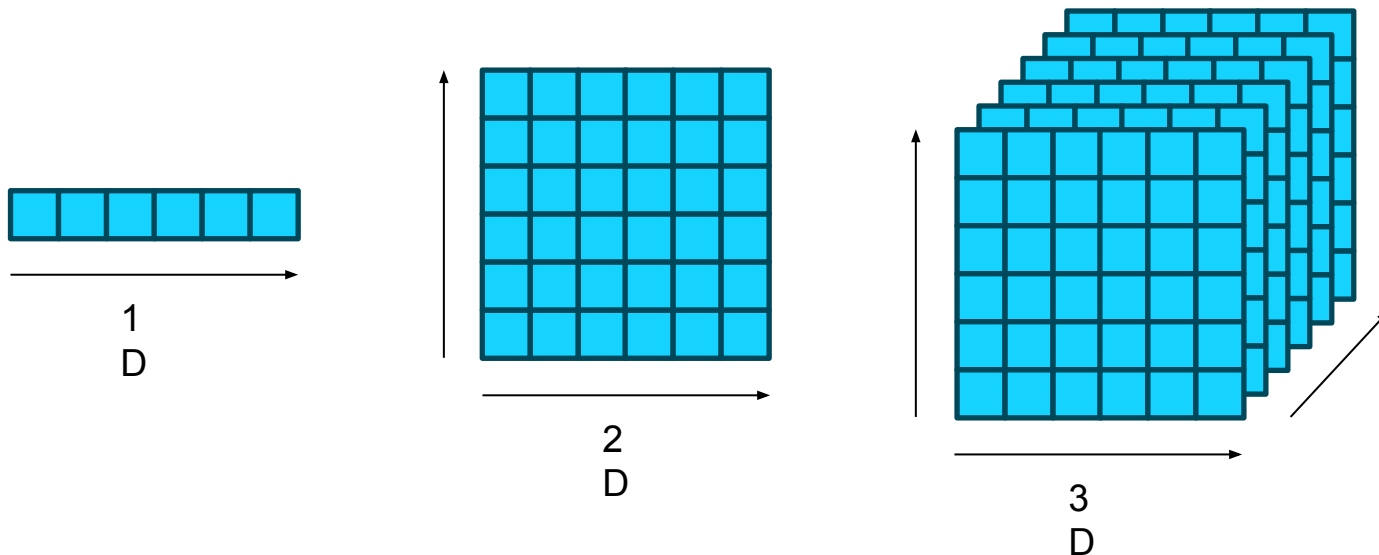
$p * q^2$ -----> Feature map 2

Agenda

1. Aplicaciones
2. Convoluciones
3. Ejemplo discreto
- 4. Convoluciones multidimensionales**
5. Completamente conectada (FC) vs. capa convolucional
6. Kernels y Filtros
7. Padding y Stride
8. Pooling
9. Arquitectura de una CNN
10. Bibliografía

Convoluciones Multidimensionales

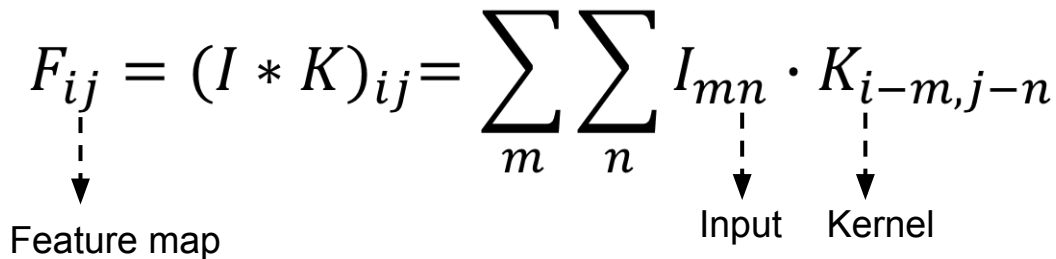
- En aplicaciones de machine learning, inputs y kernels son típicamente arrays multidimensionales (tensores)



Convoluciones Multidimensionales

- En aplicaciones de machine learning, inputs y kernels son típicamente arrays multidimensionales (tensores)
- Podemos realizar convoluciones sobre más de un eje a la vez.
 - Las convoluciones para arrays 2D se definen:

$$F_{ij} = (I * K)_{ij} = \sum_m \sum_n I_{mn} \cdot K_{i-m, j-n}$$



Feature map Input Kernel

Multidimensional Convolutions

- En aplicaciones de machine learning, inputs y kernels son típicamente arrays multidimensionales (tensores)
- Podemos realizar convoluciones sobre más de un eje a la vez.
 - Las convoluciones para arrays 2D se definen:

$$F_{ij} = (K * I)_{ij} = \sum_m \sum_n I_{i-m, j-n} \cdot K_{m,n}$$

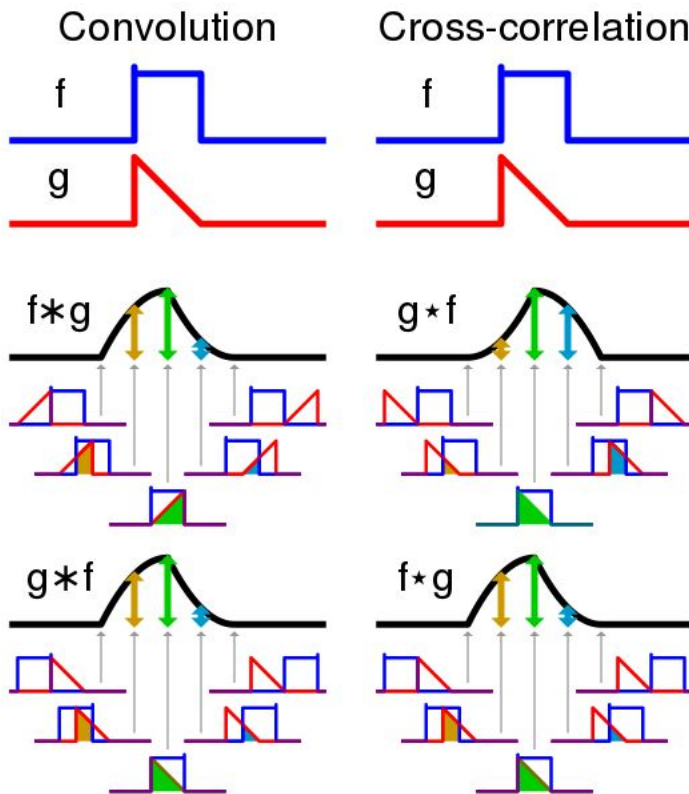
Multidimensional Convolutions

- En aplicaciones de machine learning, inputs y kernels son típicamente arrays multidimensionales (tensores)
- Podemos realizar convoluciones sobre más de un eje a la vez.
 - Las convoluciones para arrays 2D se definen:

$$F_{ij} = (I * K)_{ij} = \sum_m \sum_n I_{i+m, j+n} \cdot K_{m,n}$$

Cross-correlation

Convolución vs. cross-correlation



Ejemplo convolución 2D

Input

a	b	c	d
e	f	g	h
i	j	k	l

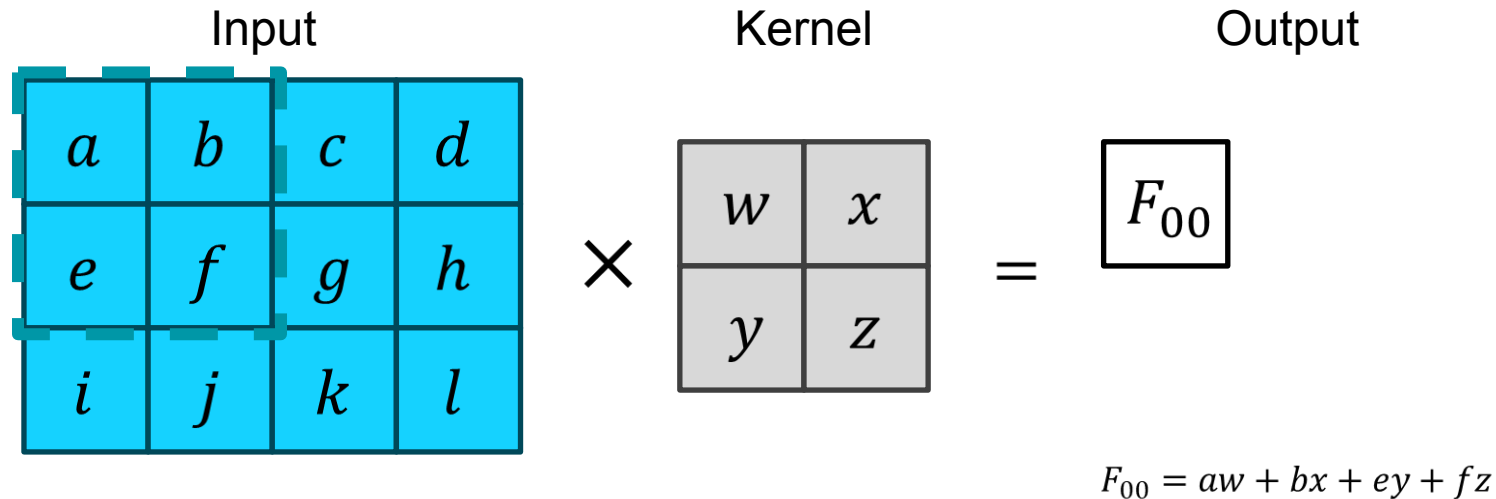
Kernel

w	x
y	z

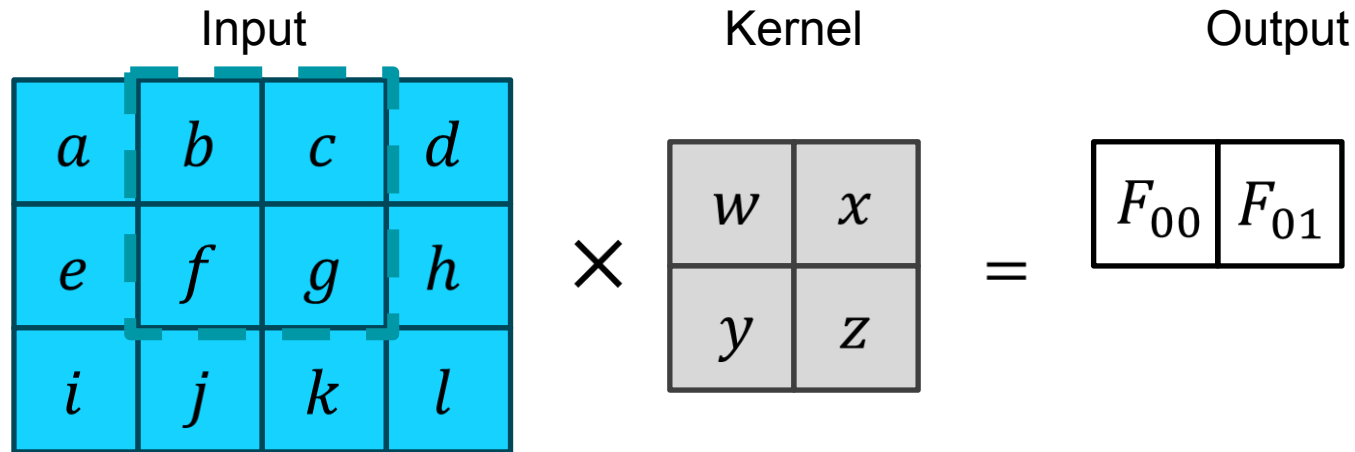
Output

$$\begin{bmatrix} a & b & c & d \\ e & f & g & h \\ i & j & k & l \end{bmatrix} \times \begin{bmatrix} w & x \\ y & z \end{bmatrix} =$$

Ejemplo convolución 2D



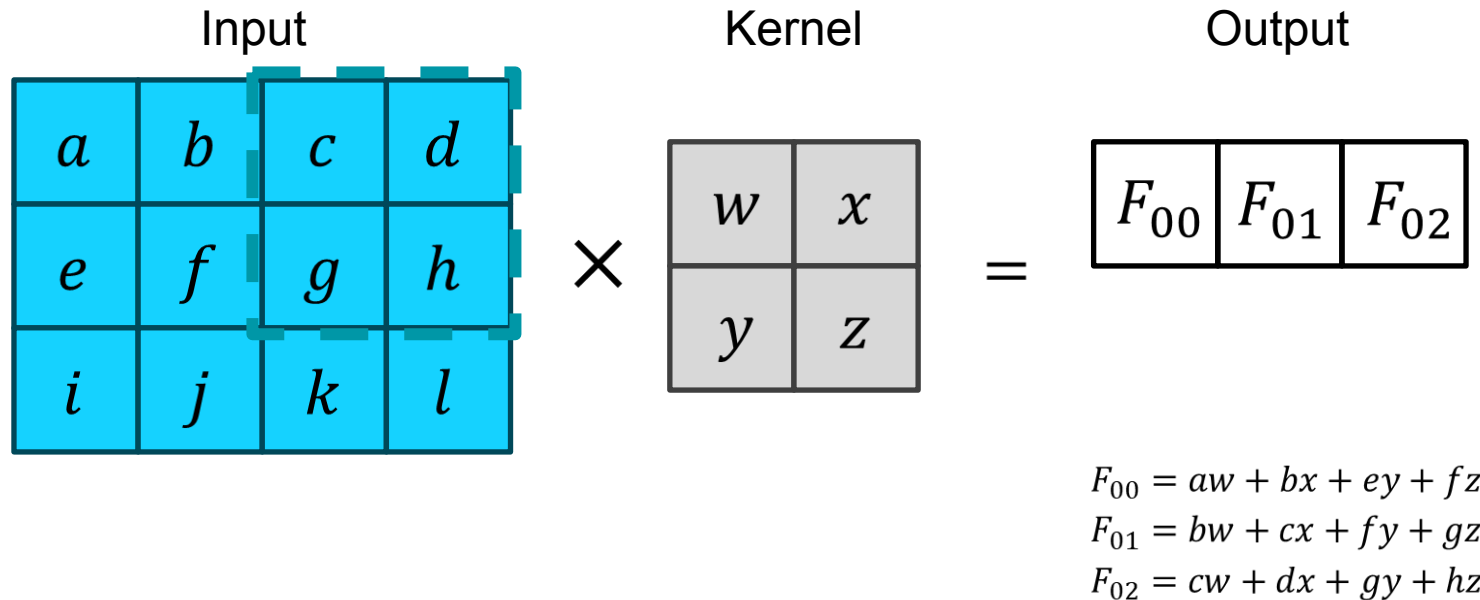
Ejemplo convolución 2D



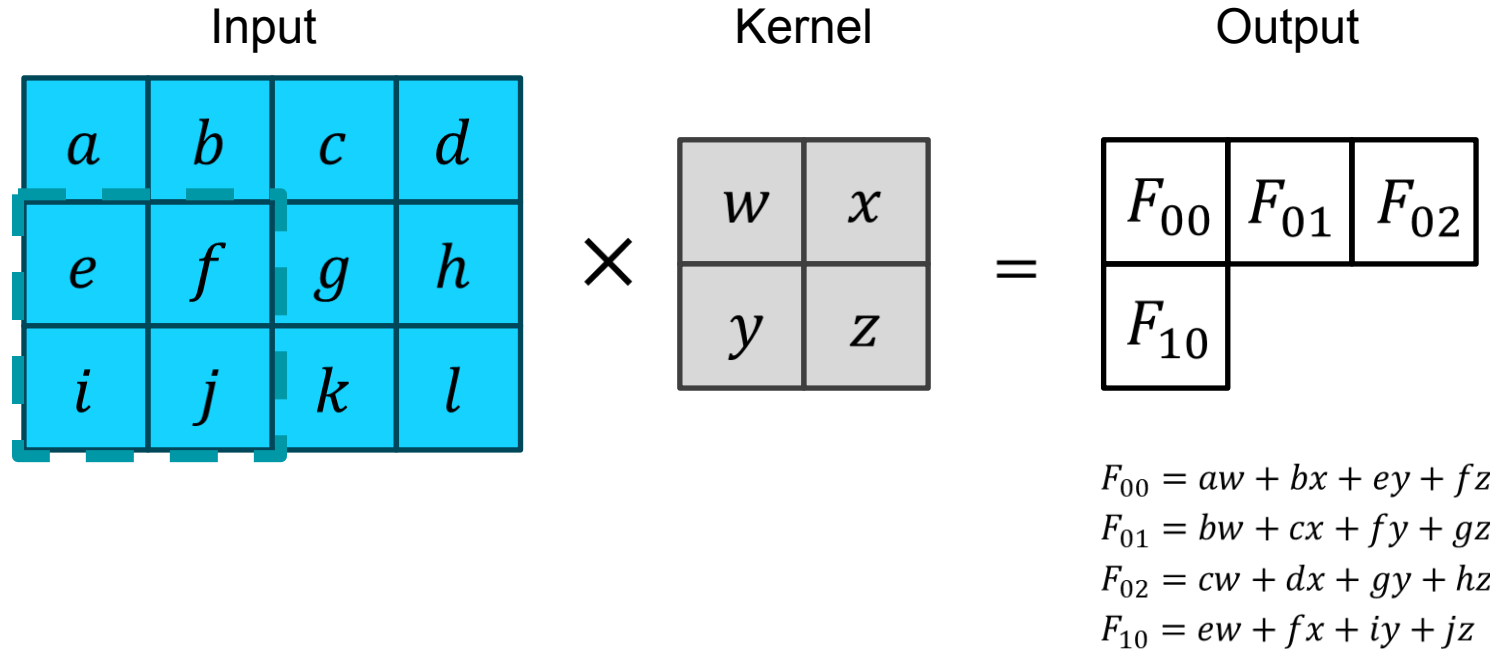
$$F_{00} = aw + bx + ey + fz$$

$$F_{01} = bw + cx + fy + gz$$

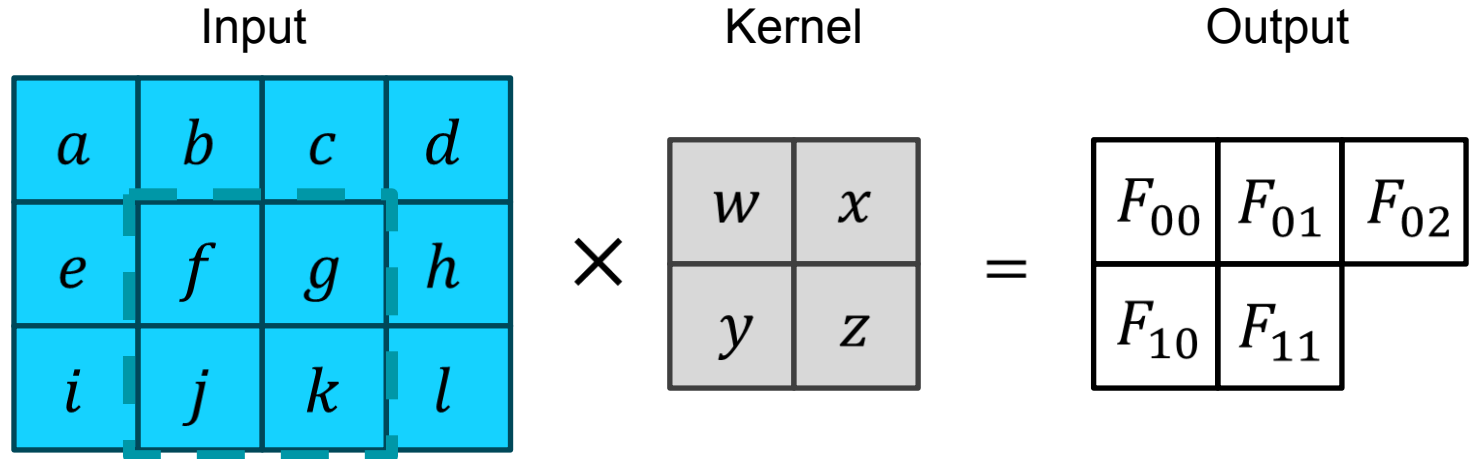
Ejemplo convolución 2D



Ejemplo convolución 2D



Ejemplo convolución 2D



$$F_{00} = aw + bx + ey + fz$$

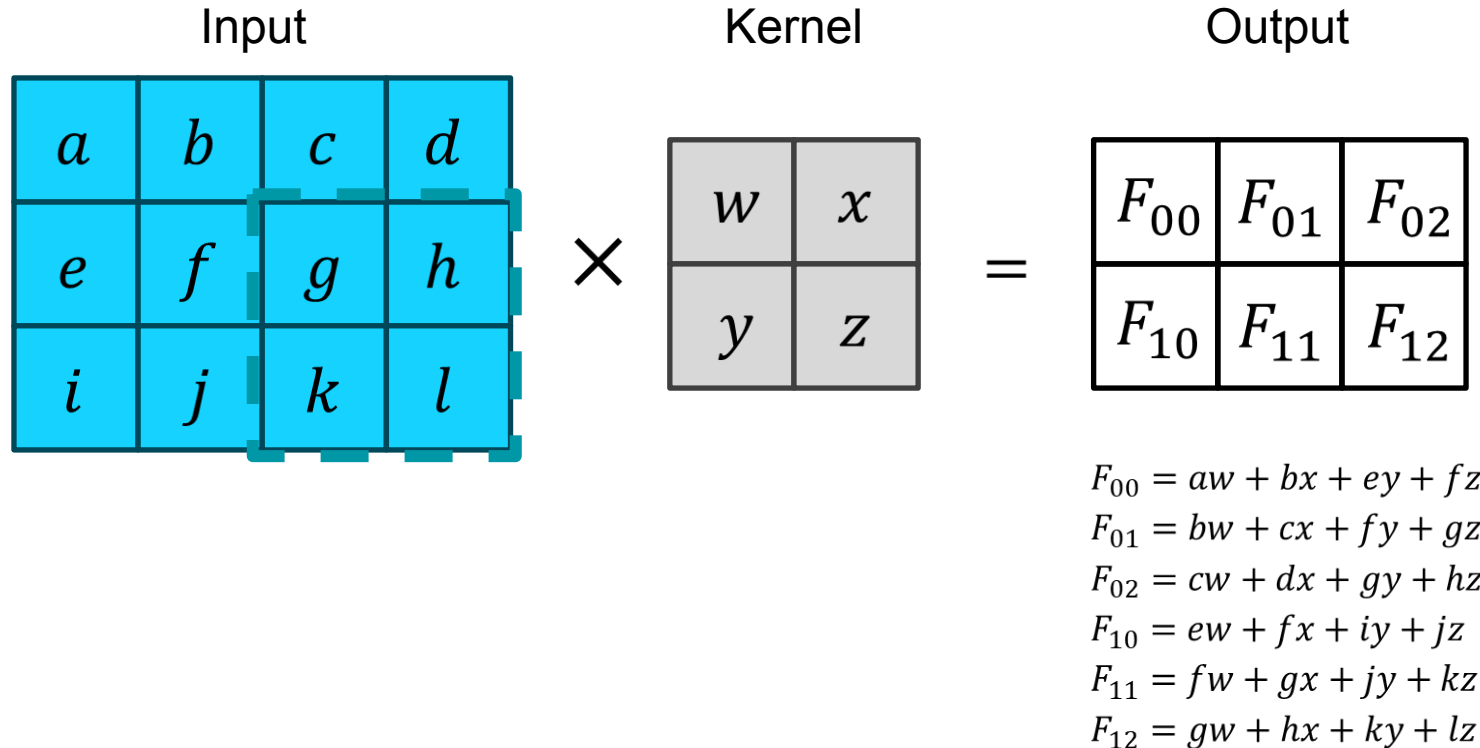
$$F_{01} = bw + cx + fy + gz$$

$$F_{02} = cw + dx + gy + hz$$

$$F_{10} = ew + fx + iy + jz$$

$$F_{11} = fw + gx + jy + kz$$

Ejemplo convolución 2D



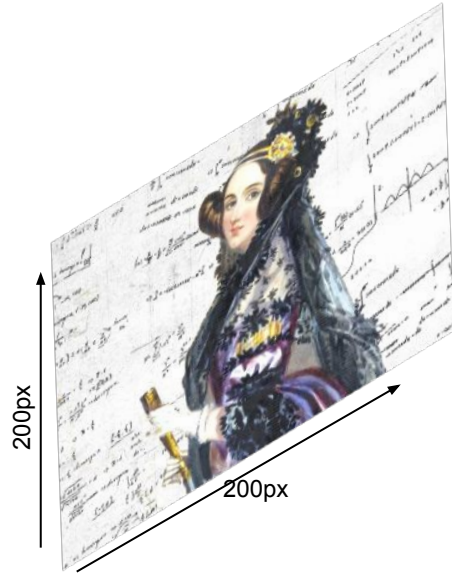
¿Por qué son útiles?

- **Correlaciones esparsas:** Los Kernels suelen ser más pequeños que las entradas; almacenamos y calculamos menos parámetros.
- **Mismos parámetros:** Los mismos parámetros de un Kernel se usan para todo el input. No necesitamos aprender nuevos parámetros para cada ubicación.
- **Equivarianza traslacional:** Propiedad de mantener relaciones consistentes entre las entradas y las salidas cuando se aplican desplazamientos o traslaciones a las entradas. Si la entrada cambia de posición, la salida cambia en consecuencia.

Agenda

1. Aplicaciones
2. Convoluciones
3. Ejemplo discreto
4. Convoluciones multidimensionales
5. **Completamente conectada (FC) vs. capa convolucional**
6. Kernels y Filtros
7. Padding y Stride
8. Pooling
9. Arquitectura de una CNN
10. Bibliografía

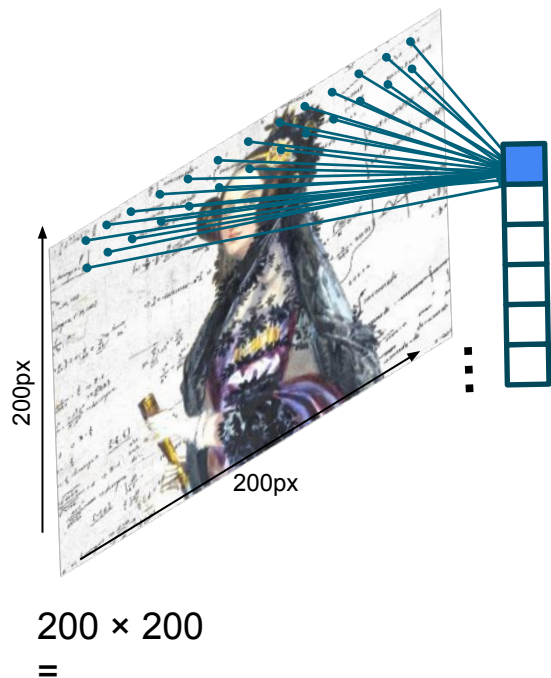
Completamente conectada (FC) vs. capa convolucional



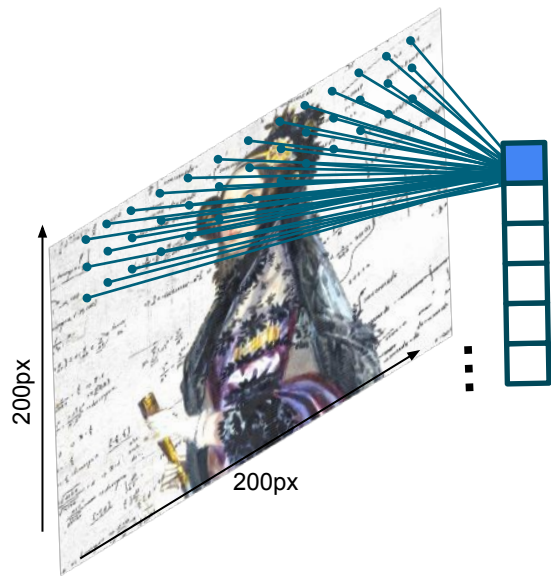
200×200

$=$

Completamente conectada (FC) vs. capa convolucional

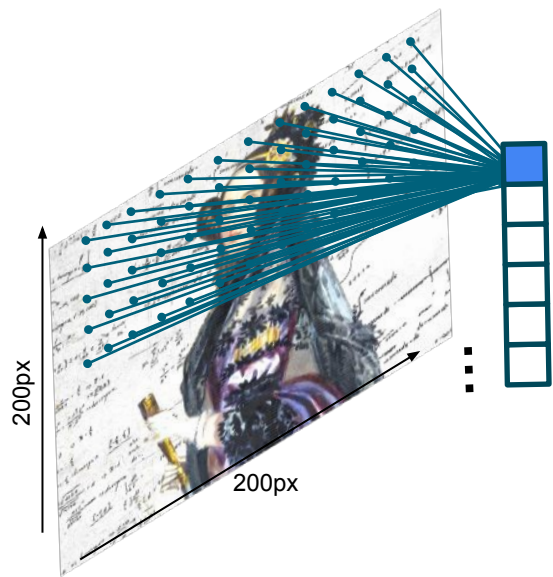


Completamente conectada (FC) vs. capa convolucional



$200 \times 200 = 40,000$ hidden units

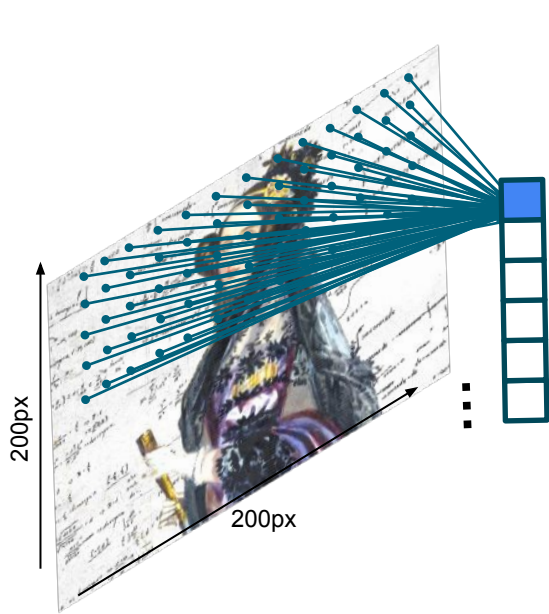
Completamente conectada (FC) vs. capa convolucional



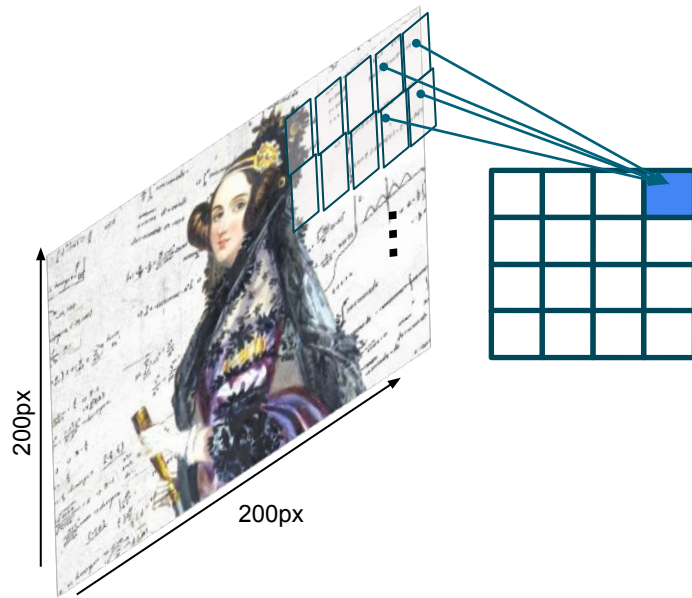
$200 \times 200 = 40,000$ hidden units

Equals approximately 1.6 billion parameters!

Completamente conectada (FC) vs. capa convolucional

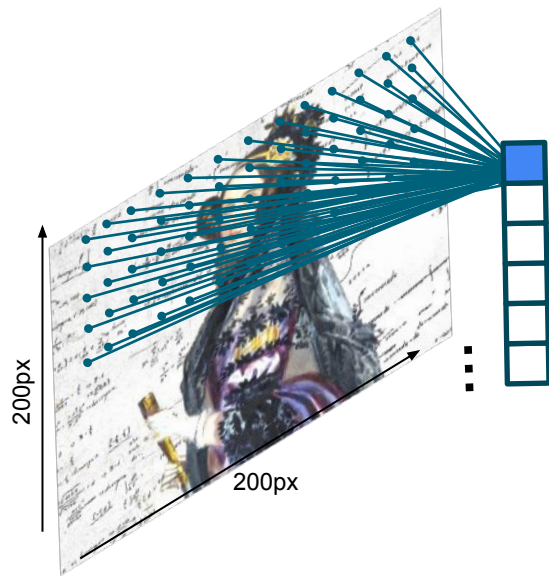


$200 \times 200 = 40,000$ hidden units

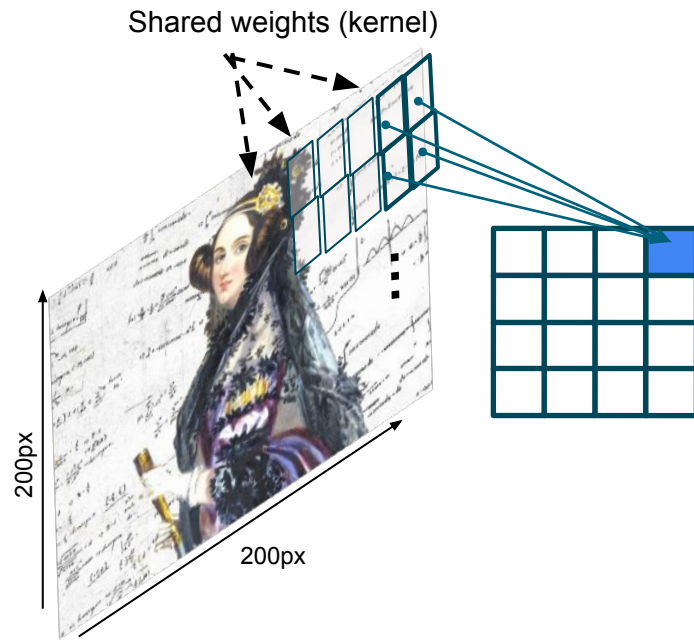


Equals approximately 1.6 billion parameters!

Completamente conectada (FC) vs. capa convolucional



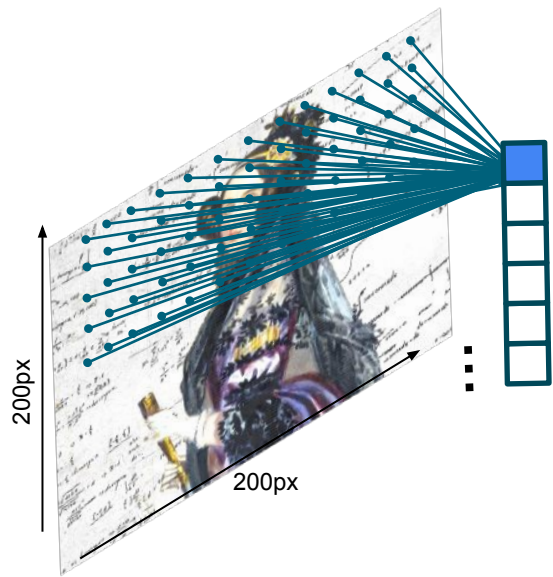
$200 \times 200 = 40,000$ hidden units



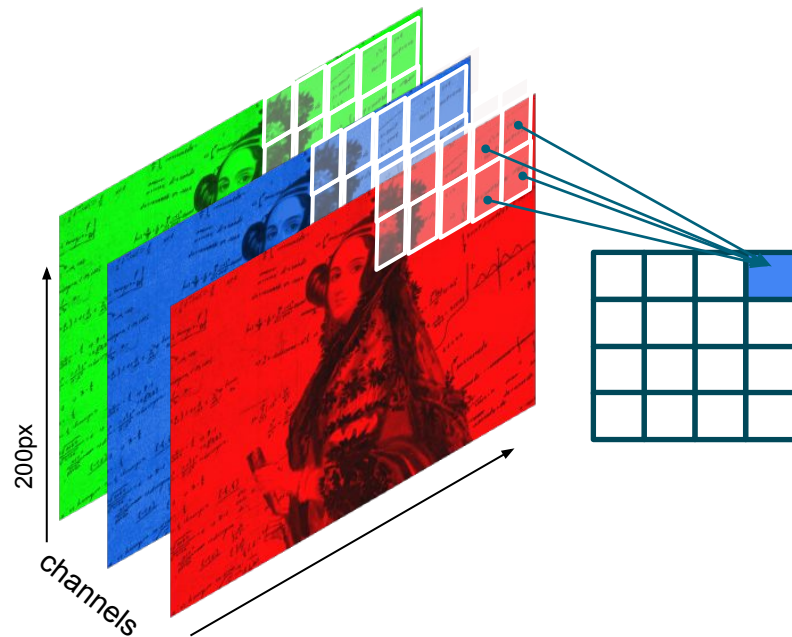
4 weights + bias

Equals approximately 1.6 billion parameters!

Completamente conectada (FC) vs. capa convolucional



$200 \times 200 = 40,000$ hidden units



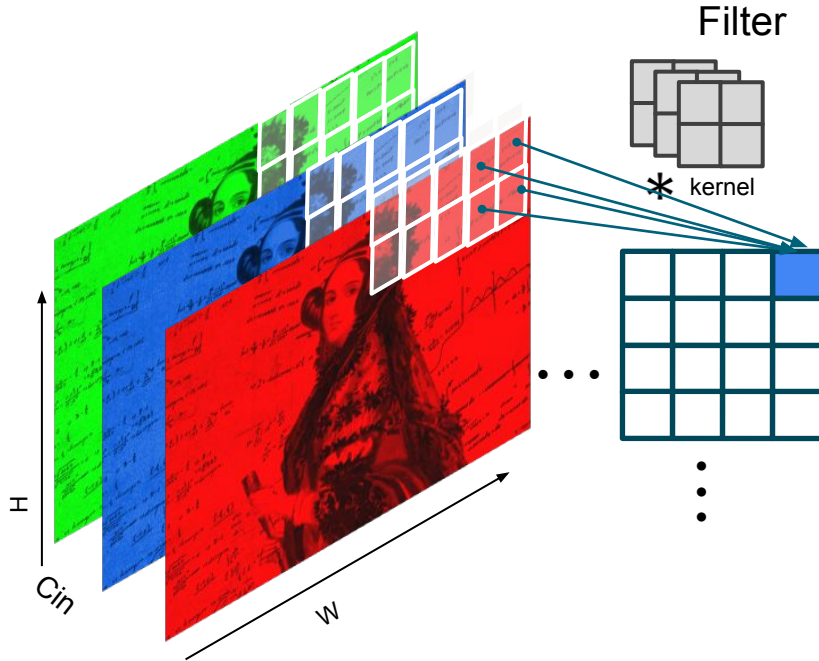
$4 \text{ weights} \times 3 \text{ (RGB channels)} + \text{bias}$
 $= 13 \text{ parameters}$

Equals approximately 1.6 billion parameters!

Agenda

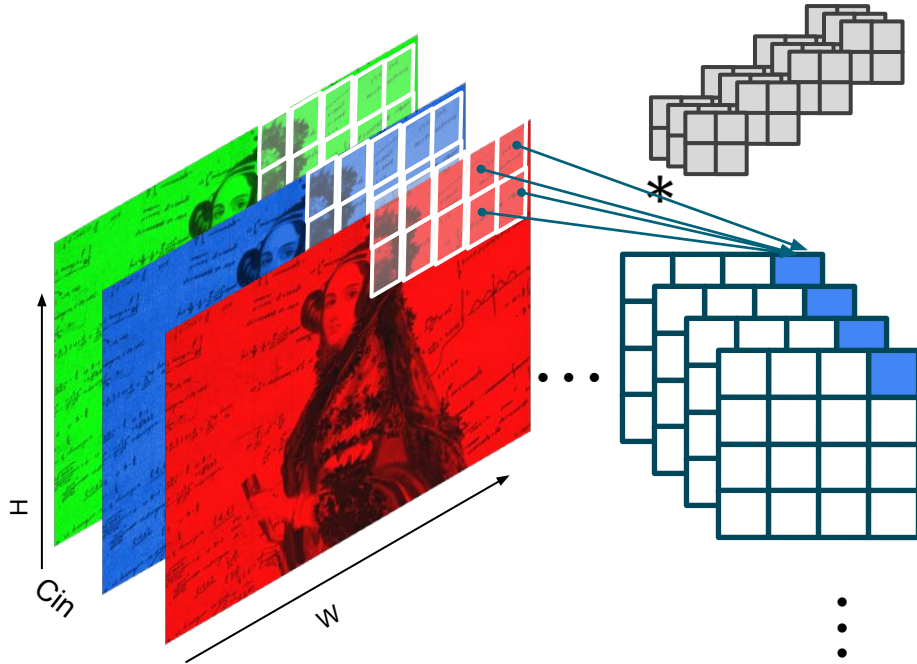
1. Aplicaciones
2. Convoluciones
3. Ejemplo discreto
4. Convoluciones multidimensionales
5. Completamente conectada (FC) vs. capa convolucional
- 6. Kernels y Filtros**
7. Padding y Stride
8. Pooling
9. Arquitectura de una CNN
10. Bibliografía

Kernels y Filtros



- En convoluciones multidimensionales, se utilizan múltiples kernels para convolucionar cada capa.
- El **filtro** es la colección de kernels utilizado en la convolución de los canales del tensor de entrada.
- Cada filtro tiene $\#kernel = \#C_{in}$
- El output depende de $\#filtros$

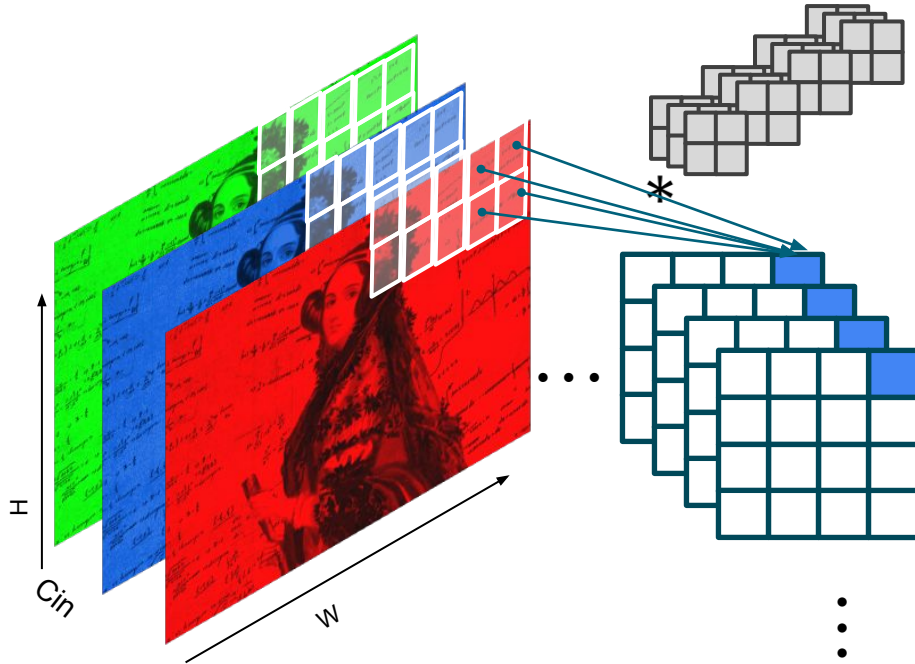
Kernels y Filtros



- Aprender muchos filtros en la entrada produce muchos mapas de salida con distintas características (feature maps)

$$F_j^n = b_j + \sum_{k=0}^{C^{in}-1} F_k^{n-1} * w_{jk}^n$$

Kernels y Filtros



- Aprender muchos filtros en la entrada produce muchos mapas de salida con distintas características (feature maps)

$$F_j^n = b_j + \sum_{k=0}^{C_{in}^{n-1}} F_k^{n-1} * W_{jk}^n$$

Diagram illustrating the convolution equation:

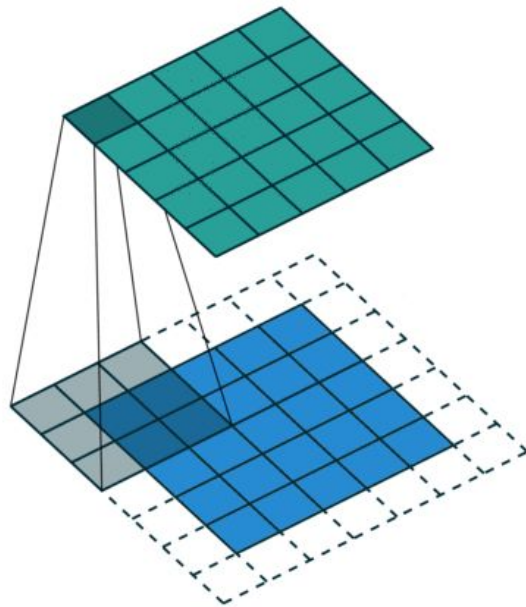
- F_j^n : Output feature maps (j)
- b_j : Bias term
- $\sum_{k=0}^{C_{in}^{n-1}}$: Sum over input feature maps (k)
- F_k^{n-1} : Input feature maps (k)
- W_{jk}^n : Kernels

Agenda

1. Aplicaciones
2. Convoluciones
3. Ejemplo discreto
4. Convoluciones multidimensionales
5. Completamente conectada (FC) vs. capa convolucional
6. Kernels y Filtros
7. **Padding y Stride**
8. Pooling
9. Arquitectura de una CNN
10. Bibliografía

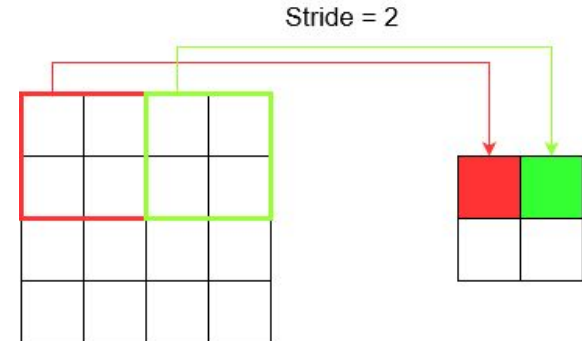
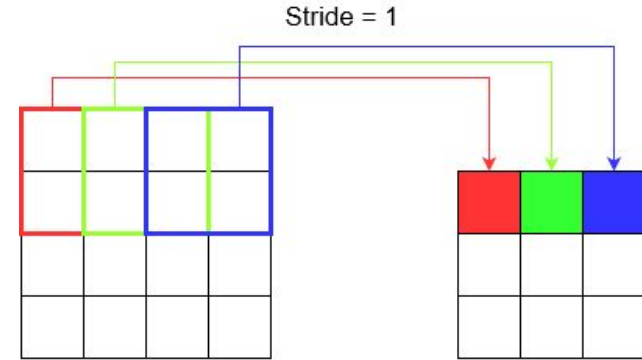
Padding

- Hiper parámetros que controlan el tamaño y forma de la salida de la capa convolucional.
- **Padding:** pixels extra que agregamos a los bordes de la imagen (matriz) de entrada; permite a la red preservar información de los bordes de la imagen.



Stride

- Hiper parámetros que controlan el tamaño y forma de la salida de la capa convolucional.
- **Stride:**
 - Controla la resolución espacial de la salida de cada capa.
 - Determina la cantidad de downsampling que ocurre en la red.

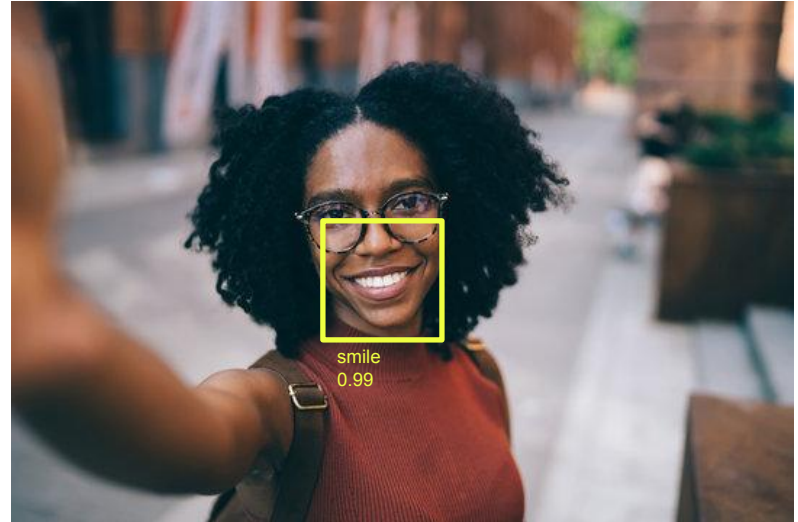


Agenda

1. Aplicaciones
2. Convoluciones
3. Ejemplo discreto
4. Convoluciones multidimensionales
5. Completamente conectada (FC) vs. capa convolucional
6. Kernels y Filtros
7. Padding y Stride
8. **Pooling**
9. Arquitectura de una CNN
10. Bibliografía

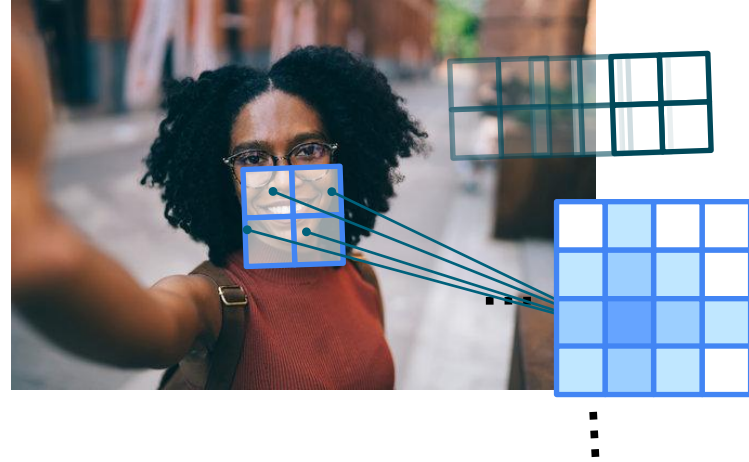
Motivación

- Estamos construyendo un detector que activa la cámara cuando sonreímos.



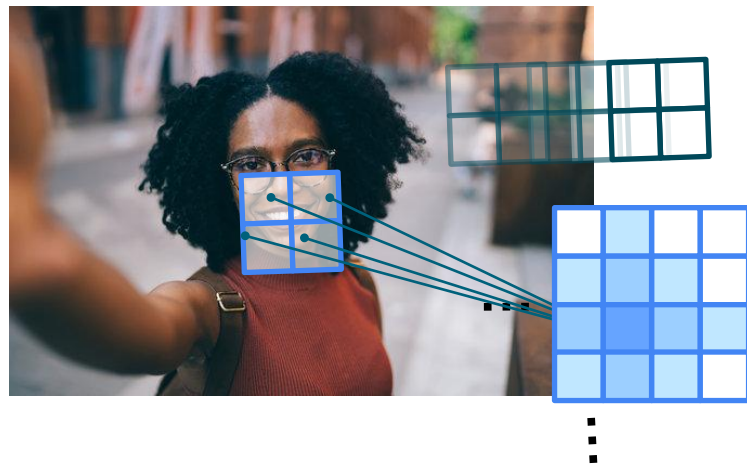
Motivación

- Aprendemos un filtro que es sensible a las sonrisas.
 - Las sonrisas pueden estar en cualquier lugar de la imagen.



Motivation

- Aprendemos un filtro que es sensible a las sonrisas.
 - Las sonrisas pueden estar en cualquier lugar de la imagen.
- ¿Cómo podemos agregar información en los feature maps para decidir si hay una sonrisa?

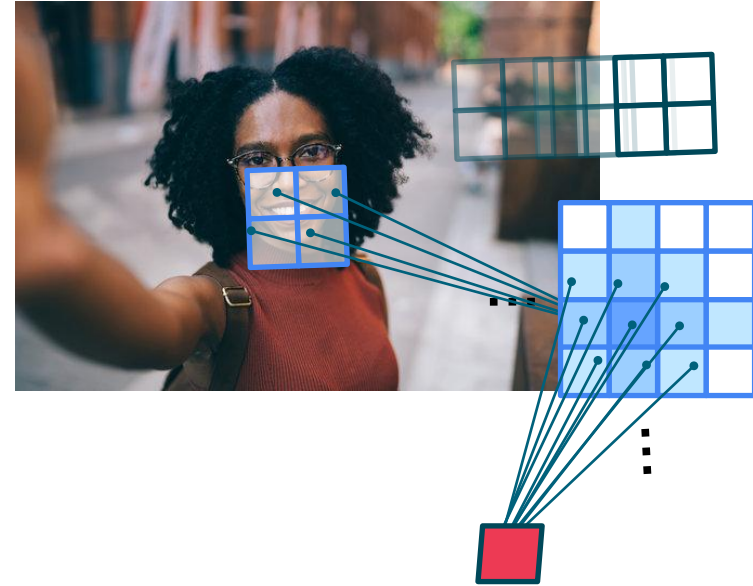


Motivation

- Una manera es realizando downsampling.
 - Una resolución más chica que todavía contiene la mayor parte o los elementos estructurales más importantes.
 - Eliminamos los detalles finos que no serían tan relevantes
- Podemos hacer downsampling con:
 - convolutional layers agrandando el stride
 - utilizando pooling layers

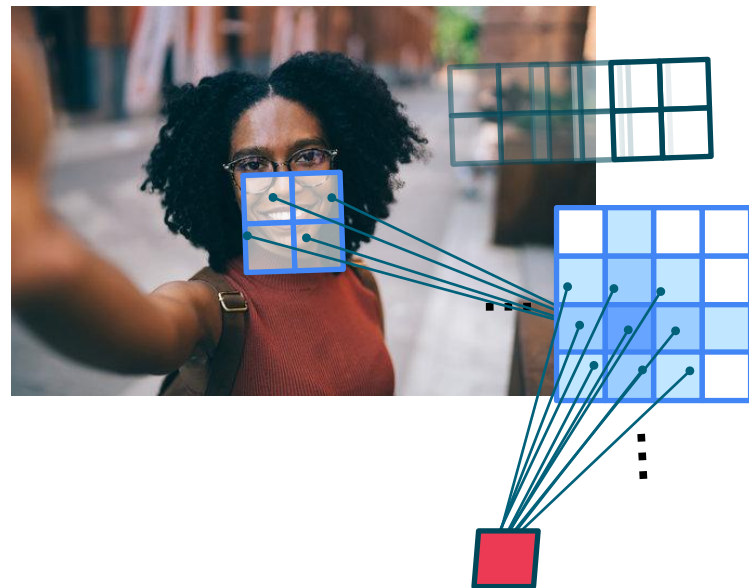
Pooling

- La función de **pooling** reemplaza la salida en una ubicación por un resumen estadístico de las salidas vecinas.



Pooling

- La función de **pooling** reemplaza la salida en una ubicación por un resumen estadístico de las salidas vecinas.
- El down-sampling no lineal se utiliza para simplificar la salida de la capa convolucional (por ejemplo, el maximum pooling consiste en tomar el valor máximo).



Max Pool

2	2	7	3
9	4	6	1
8	5	2	4
3	1	2	6

Max Pool
→
Filter - (2 x 2)
Stride - (2, 2)

9	7
8	6

Avg Pool

2	2	7	3
9	4	6	1
8	5	2	4
3	1	2	6

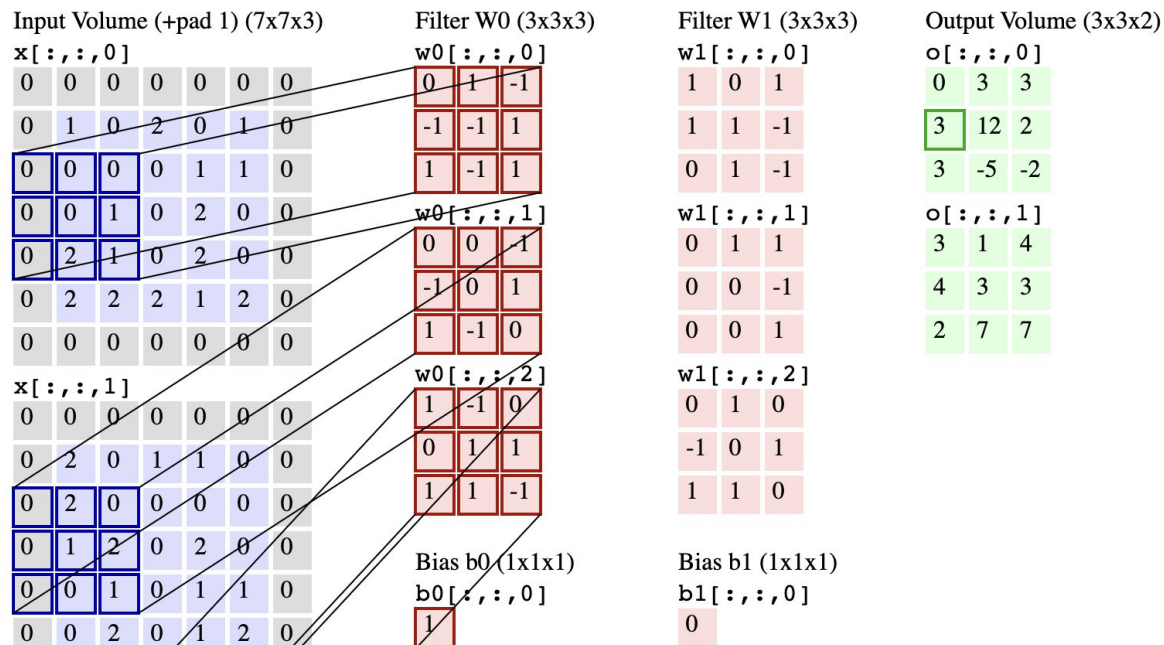
Average Pool
→

Filter - (2 x 2)
Stride - (2, 2)

4.25	4.25
4.25	3.5

Pooling Benefits

- Robusto a traslaciones
- Reduce el cálculo para las próximas capas
 - reduce el tamaño de los feature maps
- Las más comunes son
 - Max pooling
 - Average pooling



<https://cs231n.github.io/convolutional-networks/>

toggle movement

$$F_j^n = b_j + \sum_{k=0}^{C^{in}-1} F_k^{n-1} * w_{jk}^n$$

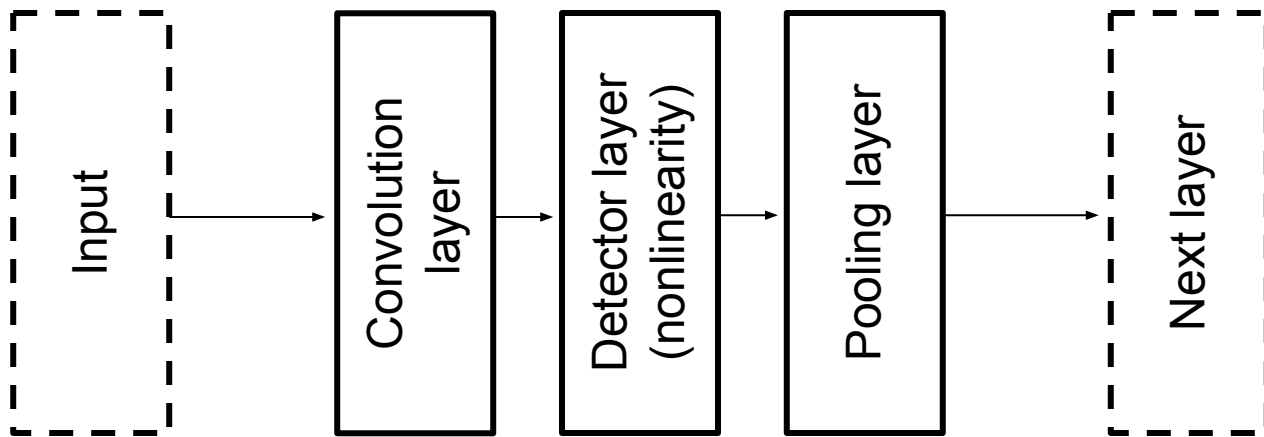


Agenda

1. Aplicaciones
2. Convoluciones
3. Ejemplo discreto
4. Convoluciones multidimensionales
5. Completamente conectada (FC) vs. capa convolucional
6. Kernels y Filtros
7. Padding y Stride
8. Pooling
- 9. Arquitectura de una CNN**
10. Bibliografía

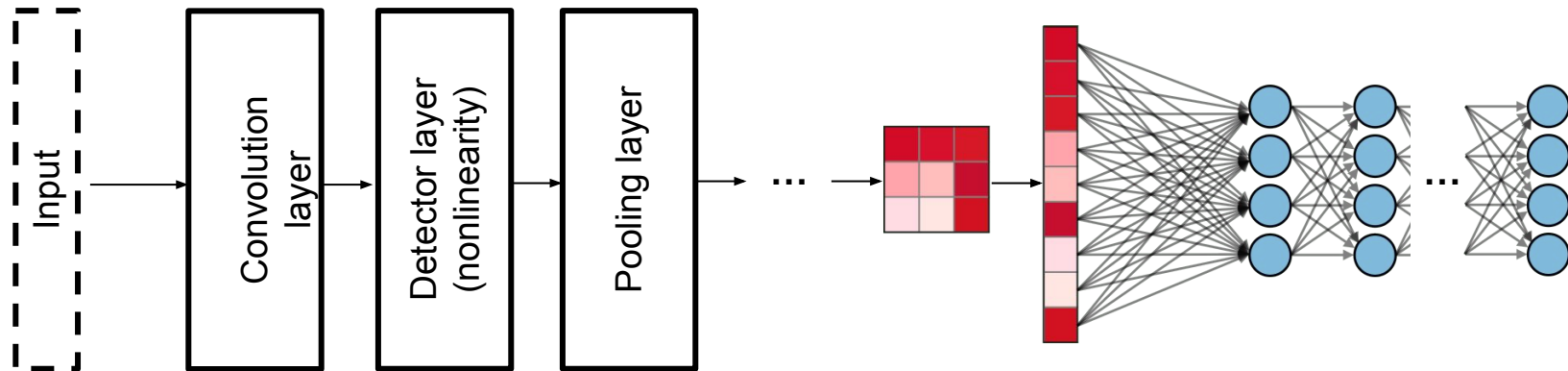
Arquitectura de una CNN

- Componentes en capas de una red neuronal convolucional típica.



Arquitectura de una CNN

- Muchas veces vamos a encontrar capas completamente conectadas (FC) hacia el final de una arquitectura CNN
 - Se usan para optimizar funciones de objetivos para las distintas tareas (ej. clasificación)



Arquitectura de una CNN

- Apilar varias capas convolucionales permite que la red aprenda representaciones jerárquicas de los datos

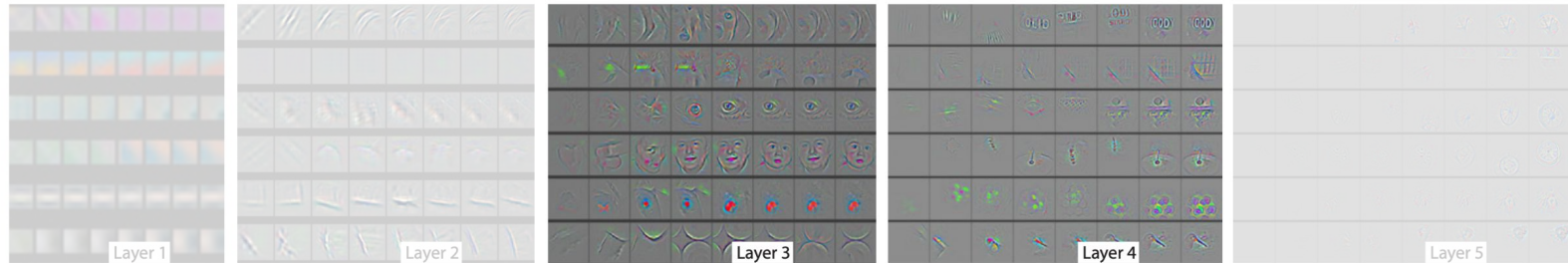
Filtros primeras capas

- Las capas inferiores tienden a aprender filtros para bordes o direcciones (es decir, líneas horizontales/verticales)



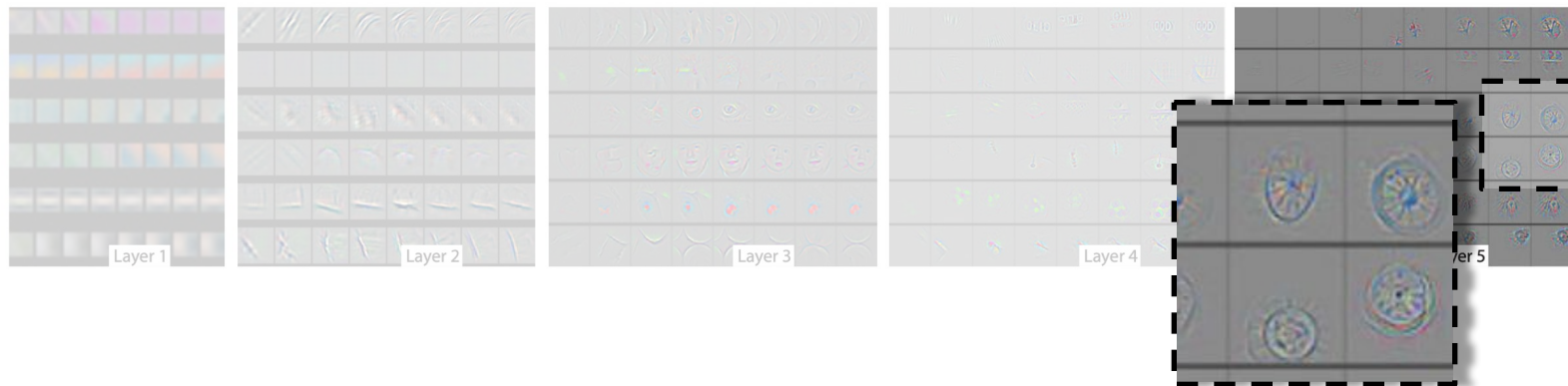
Filtros capas intermedias

- Las capas intermedias aprenden filtros más informativos para patrones básicos (es decir, texturas y formas simples).



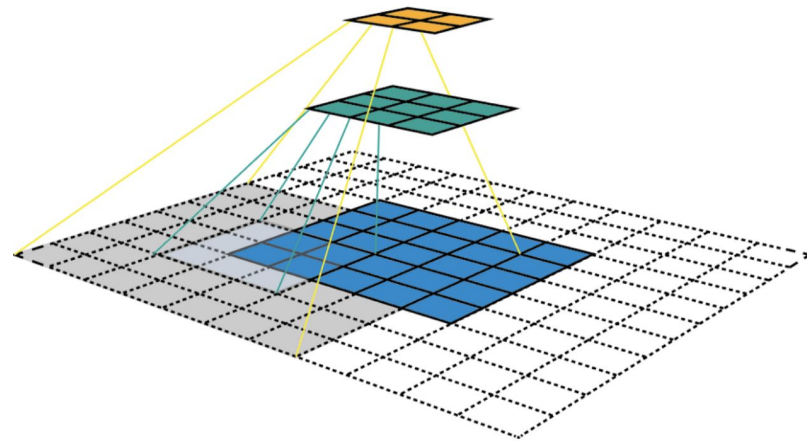
Filtros últimas capas

- Las capas superiores detectan patrones que se asemejan a partes completas de objetos y tienden a ser rasgos únicos para esa clase/objeto.



Campo Receptivo (Receptive Field)

- Es el área que capta cada unidad de la entrada.
- El tamaño y ubicación de campo receptivo afecta la habilidad de la red de detectar patrones en los datos de entrada.

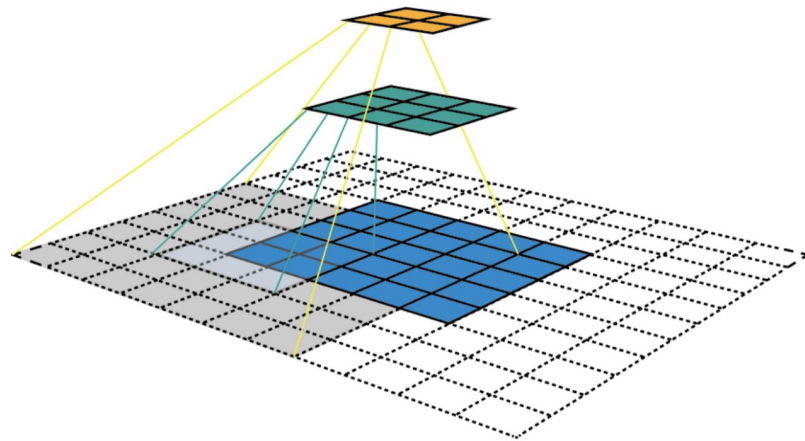


Campo Receptivo (Receptive Field)

- **Campos Receptivos:**

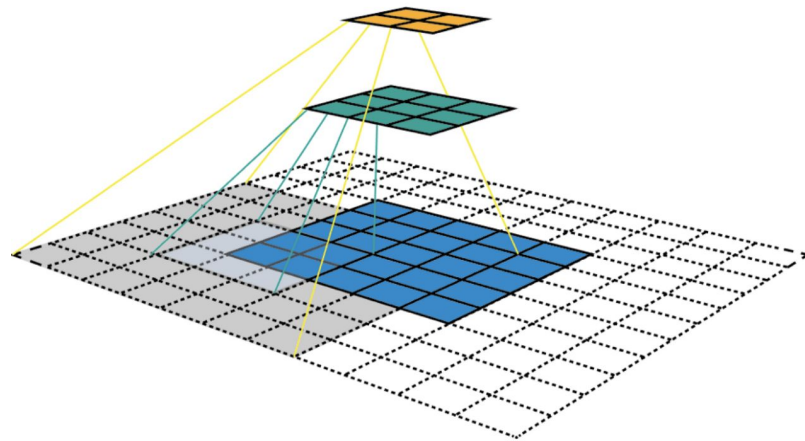
- en las capas iniciales las neuronas tienen campos receptivos locales y pequeños. Las neuronas en estas capas detectan características simples como bordes o esquinas.
- A medida que se avanza en la red, las neuronas tienen campos receptivos más grandes. Las neuronas en estas capas detectan patrones más complejos y partes de objetos.

- **Jerarquía:** Las neuronas en capas superiores tienen campos receptivos que abarcan una parte más grande de la entrada y capturan características más abstractas y de alto nivel. Esta jerarquía permite que la red reconozca objetos y patrones complejos.



Campo Receptivo (Receptive Field)

- **Stride y Pooling:** El tamaño del campo receptivo está influenciado por factores como el tamaño del filtro, stride y pooling.
- **Stride** determina cuánto se desplaza el campo receptivo con cada capa, y el **pooling** reduce las dimensiones espaciales pero aumenta el tamaño del campo receptivo.



Agenda

1. Aplicaciones
2. Convoluciones
3. Ejemplo discreto
4. Convoluciones multidimensionales
5. Completamente conectada (FC) vs. capa convolucional
6. Kernels y Filtros
7. Pooling
8. Padding y Stride
9. Arquitectura de una CNN
10. **Bibliografía**

Bibliografía

- [Deep Learning](#), Ian Goodfellow and Yoshua Bengio and Aaron Courville - Capítulo 9
- Neural Networks and Deep Learning. Charu C. Aggarwal 8.1 8.2
- Lecun, Y.; Bottou, L.; Bengio, Y.; Haffner, P. (1998). "Gradient-based learning applied to document recognition" (PDF). Proceedings of the IEEE. 86 (11): 2278–2324. doi:10.1109/5.726791. S2CID 14542261.

http://vision.stanford.edu/cs598_spring07/papers/Lecun98.pdf